AutoBayes against brms's syntax: an audit

Prepared for Andrew

24 October 2026

You asked: *"I'm trusting that the code we have really matches a lot of the brms syntax (without extraneous things)?"* This report answers with a corpus of 248 brms formulas, each read by brms 2.23.0 itself and by AutoBayes on `main` (`a72647e`), and with a review of the call surface, the post-processing names and everything the package and the repository ship beyond a brms-like library. Nothing under `autobayes/` was changed.

1 The short answer

What AutoBayes accepts, it reads as brms does, with one exception. Of the 245 corpus formulas that brms accepts, AutoBayes builds 116. For 114 of them its model has exactly brms's reading: the same design columns, the same group-level partitions, covariates and level labels, the same addition-term and special-term data, the same centring, family and links (Section 2 says how this is checked). One differs in a documented way (`family = Gamma()` from R's `stats` package has the inverse link, `ab.gamma` the log link, D9). One is a **silent misreading**: an ordered factor predictor, `y ~ fo`, is coded with treatment dummies where brms uses R's polynomial contrasts (`fo.L`, `fo.Q`). Nothing crashed.

It refuses about half of brms's syntax, nearly always by name. The other 129 formulas are refused: 113 with a message naming the construct, 16 with a message that does not name it or names it wrongly (Table 2). The worst of these is `(x | g)`: AutoBayes says it has no varying intercept, but brms's `(x | g)` *is* `(1 + x | g)`. The core of brms's language is covered and matches: numeric and factor terms, interactions with their main effects, cell-means coding, intercept controls, the supported transforms, offsets, `(1 | g)`, `||`, `|`, `g:h`, `g/h`, `gr(g, by = )`, `mm()`, distributional and non-linear formulas, `mvbind()` with `set_rescor()`, `mi()`, `me()`, `mo()`, `cs()`, `gp()`, `ar()`/`ma()`/`arma()`, `weights()`, `trials()`, `cens()` and `trunc()`. What is refused includes smooths (`s()`, `t2()`), `se()`, `rate()`, `thres()`, most `gr()` options and several cheap spellings of things AutoBayes already has (`gr(g, cor = FALSE)` for `||`, `trials(10)`, `(x | g)`).

It is not free of extraneous things. The top-level namespace exports a reproducibility and diagnostics API from the agent-scaffold era (`environment`, `fit_record`, `compare_records`, `format_comparison`, `format_environment`, `panel`) and the term model's internals (`ModelSpec`, `PopTerm`, `GroupTerm`, `Categorical`, `parse_formula`, `NonlinearSpec`); user-facing text carries project jargon (`describe()` prints "AutoBayes M0 model"; 36 of the 132 refusal messages in the corpus say "this slice" or "this subset", 17 cite a milestone id, 8 a milestone decision document); and the repository still carries the agent scaffold (`SKILL.md`, `skills/`, `scaffold/`, `reference/`, `bin/`, `install.sh`, `cluster/`) and 2.8 MB of milestone evidence. The library does not read any of it at run time (Section 6). Recommendations, ranked, are in Section 7.

2 How the corpus was read

The corpus. `tests/brms_syntax/corpus.json` has 248 entries in ten areas (Table 1). Each holds a brms call, written for this audit from brms 2.23's help pages (`brmsformula`, `addition-terms`, `brmsformula-helpers`, `gr`, `mm`, `mmc`, `gp`, `s`, `mo`, `me`, `mi`, `cs`, the autocorrelation pages, `mvbrmsformula`, `brmsfamily`, `mixture`, `custom_family`); no brms source was read. Each also holds the AutoBayes translation a user would write: the constructor for the family, the same formula string (a list of strings for `bf()` with several formulas), `categorical={column: first level}` for a factor predictor, and `bf()`'s options as keyword arguments (`center` as `center_predictors`, `nl`, `loop`, `quantile`; any other option under its brms name). Priors are `ab.default_priors()` for the same formula, so the formula is the only thing under test. The data are one seeded 40-row frame with 60 columns (`audit.make_data`, committed as `data.csv`): numeric, integer, logical, text, factor, ordered-factor and pandas-categorical columns, five grouping levels with eight time points each, counts, trials, ordinal and categorical responses, censoring codes, weights and a column with missing values.

brms's reading. `tests/brms_syntax/brms_reading.R` builds each entry's formula and family with `as.formula()` and brms's helpers (no text is evaluated as code) and calls `make_standata()` and `brm(..., empty = TRUE)`, which builds the fit object without compiling, for brms's table of group-level terms and its resolved family and links. The output is `standata.json` (0.8 MB): every Stan-data name with its dimensions, the arrays, the design matrices' column names, the group-level terms with their level labels, and the family. Reading all 248 entries takes about 25 seconds, and the output is the same byte for byte on a second run.

The comparison (`audit.compare`). An accepted model matches brms when

- every AutoBayes design column equals exactly one column of brms's `X` (or `X_sigma`, `X_y`, `X_a`, `X_muc2`, `X_mu1`, ...) to 10^{-12}, and brms's only other column is its intercept;
- every group-level coefficient meets exactly one brms term whose `J` induces the same partition of the rows and whose `Z` is the same covariate (`J_m_k`, `W_m_k` per member for `mm()`; `Jby` for `by`), with the same level labels and the same correlation flag;
- trials, offsets, weights, censoring codes and bounds, `Xmo`/`Jmo`, `Xcs`, `Jgp`/`Xgp` and the GP scaling, `Xn`/`noise`, `Jmi`, `Kar`/`Kma`/`J_lag`, `nthres`, `ncat` and a fixed `quantile` agree;
- brms centres the design (`Kc`) exactly when AutoBayes does, and the family and every link agree;
- *no brms Stan-data name is left over*: a name the AutoBayes model does not account for (`se`, `denom`, a fixed `sigma`, `X_mu`, ...) is a mismatch. This is what would catch a construct read and silently dropped.

Group-level labels in another order (brms orders integer labels numerically, AutoBayes as text, D3) are recorded as a note, not a mismatch. On 43 deliberate mismatches (each model against a neighbouring entry's brms reading: an extra column, `|` against `||`, exact against approximate GP, centred against uncentred, logit against probit, ...) the comparison reported a difference every time.

Refusals. A refusal is AutoBayes's own `FormulaError`, `PriorError`, `ValueError` or `NotImplementedError` raised in `autobayes/`, or Python's `TypeError` for a keyword the constructor does not have; anything else would be a crash. Each refusal message was read; `tokens` records what a message naming the construct contains, and `unclear` says why one that quotes the construct still misnames it.

Area	entries	match	differs	misread	refused named	refused unclear	brms refuses
population-level terms	73	39		1	26	7	
group-level terms	35	13			17	5	
addition terms	31	11			20		
special terms (`s`, `gp`, `mo`, `me`, `cs`)	25	9			16		
autocorrelation terms	13	6			7		
`bf()` options and helpers	12	6			6		
distributional formulas	14	7			5	2	
non-linear formulas	10	6			3	1	
multivariate formulas	10	3			6	1	
families and their responses	25	14	1		7		3
total	248	114	1	1	113	16	3

Table 1: Outcomes by area. "brms refuses": brms stops (`k ~ x` for a binomial without trials; a text response for `cumulative`; `cox()`, which needs the uninstalled `splines2`); AutoBayes refuses those three by name too. No entry crashed.

The test. `tests/test_brms_syntax_corpus.py` (250 tests, 5 s, no R) rebuilds every entry: an accepted formula must still match brms's reading, a refusal must still name its construct, and each known gap is pinned as it is today. Every test id starts with the entry's gap: `defect:` (a misreading, an unclear refusal or a crash: a fix shows up as a failure of `test_known_gap_is_pinned`), `capability:` (refused by name), `divergence:` (documented) or `ok:`. `python -m tests.brms_syntax.audit` re-records the AutoBayes side.

3 The formula language

3.1 The silent misreading, and the documented difference

Ordered factors as predictors (`pop_ordered_factor`). With `fo` an ordered factor (`lo` < `mid` < `hi`), brms's `y ~ fo` has the columns `Intercept`, `fo.L`, `fo.Q`: R's `contr.poly`, orthogonal linear and quadratic contrasts over the three ordered levels. AutoBayes, given the pandas ordered Categorical with `categorical={"fo": "lo"}`, builds `fo[hi]` and `fo[mid]`, treatment dummies against `lo`, and says nothing. The two designs span the same column space, so the likelihood is the same family of models, but the coefficients mean different things: brms's `b_fo.L` is a linear trend across the levels, AutoBayes's `fo[hi]` the difference of `hi` from `lo`. A prior on `b`, a `hypothesis()` on a coefficient or a reported estimate therefore refers to another quantity than the brms model it was ported from. No divergence row or document mentions it. (An unordered factor, and a pandas Categorical declared with its first level, match brms exactly.)

The Gamma family's link (`fam_gamma_stats`, D9). A brms user who writes `family = Gamma()` gets the `stats` package's family, whose default link is `inverse`; `ab.gamma` defaults to `log`. That is D9. But D9 (and `ab.gamma`'s docstring) say "`brmsfamily("Gamma")` defaults to the inverse link", which is wrong: `brmsfamily("Gamma")` defaults to `log` (its help page: "when calling the `Gamma` family function of the `stats` package, the default link will be `inverse` instead of `log` although the latter is the default in brms"), and the corpus entry `fam_gamma_default_link` matches. Only the documentation needs fixing.

brms formula	AutoBayes says	What is wrong with it
`y ~ x + (x | g)`, `(x || g)`	"varying slopes without the varying intercept are not supported; write `(1 + x | g)`"	brms's `(x | g)` has the intercept: it is `(1 + x | g)`, which AutoBayes takes
`(1 | g) + (0 + x | g)`	refuses `(0 + x | g)`, advises `(1 + x | g)`	that adds a correlation; brms's reading is uncorrelated, AutoBayes's `(1 + x || g)`
`(1 + x |p| g) + (0 + z |p| g)`	the same slope-only rule, advises `(1 + z | g)`	a second term of `g`, refused in turn; brms's reading is one block, `(1 + x + z | g)`
`(x - 1 | g)`	"`'x - 1'` is not a column name"	intercept removal in a group term
`y ~ 0 + Intercept + x`	"columns not in data: `['Intercept']`"	brms's reserved `Intercept`
`y ~ splines::ns(x, df = 3)`	"only a two-way interaction is supported ... names 3 terms"	reads `::` as interaction colons
`y ~ (x + z + w)^2`	"parenthesised term is not a `(1 | group)` term"	the `^` expansion
`y ~ x %in% f`, `y ~ .`, `y ~ ` `` `x.dot` ``	"... is not a column name"	the operator, R's dot, backticks (the column `x.dot` exists)
`log(ypos) ~ x`, `cbind(y, y2) ~ x`	"expected `'response ~ terms'`"	a transformed or matrix response
`bf(yord ~ x, disc ~ z)` (cumulative)	"the cumulative family has no parameter `'disc'`"	brms's has one (fixed at 1; D38 is the real reason)
`bf(y ~ x) + nlf(sigma ~ a * exp(c * z))`	refuses `a ~ 1`: "the gaussian family has no parameter `'a'`"	the non-linear formula of `sigma`
`+ nlf(a ~ c * z)` (nested)	"`'c'` does not appear in the non-linear formula"	a nested non-linear parameter (D60)

Table 2: The 16 unclear refusals (`defect:` in the test ids). Each is a refusal, so none is unsafe; each would send a brms user the wrong way.

3.2 Refusals that do not name the construct

Table 2 lists them. Besides these, six refusals name the term but give a generic reason: `s(x)`, `s(x, k = 5)`, `s(x, by = f)`, `s(x, z)`, `t2(x, z)` and `sigma ~ s(z)` get the transforms message ("only `log(col)`, `exp(col)`, ... are supported as in-formula transforms ... compute the column in the data frame"), which treats a smooth as a transform and gives advice that cannot apply; `(cs(1) | g)` gets "function calls inside a group term are not supported". They are counted as named.

3.3 What matches, and what is refused by name

Accepted and identical to brms's reading (114): `y ~ x`, `x + z`, `1`, `1 + x`, `x + 1`, `0 + x`, `x - 1`, `-1 + x`, `x + 0`; a factor (declared) alone, with a numeric term, two factors; cell-means coding `0 + f`, `0 + f + x`, `0 + x + f`, `0 + f + f2`, `0 + f * x`; a factor with an unused declared level; brms's `factor(o)` written as `categorical={"o": 0}`; `x * z`, `x + z + x:z`, `z:x`, `x * f`, `f * f2`; `I(x^2)`, `I(x^3)`, `I(2 * x)`, `I(x - 1)`, `log()`, `exp()`, `sqrt()`; `offset(col)`, `offset(log(col))`; a dotted column name; no spaces, line breaks and extra spaces; `(1 | g)`, `1 + (1 | g)`, `(1 | g) + (1 | h)`, `(1 + x | g)`, `(1 + x || g)`, `(1 + x + z | g)`, `(1 | g:h)`, `(1 | g/h)`, `gr(g)`, `gr(g, by = gby)`, `mm(g, g2)` with and without weights, integer group labels (levels in another order, D3); `weights()`, `trials()`, `trials() + weights()`, `cens(c)`, `cens(c, y2)`, `weights() + cens()`, Weibull `cens()`, `trunc()` with one or both numeric bounds, count `trunc(ub = )`; `gp(x)`, `gp(x, k = 10, c = 5/4)`, `gp()` of repeated inputs, `mo()` of integers and of an ordered factor, `x + mo(o)`, `me(x, sdx)`, `cs()` in `sratio` and `acat`; `ar()` (named, positional, `p = 2`, one series), `ma()`, `arma()`; `bf()`, `center =`

`FALSE` (with groups too), `quantile = 0.25`, `+ lf(sigma ~ z)`, `+ set_nl()`; `sigma ~ z`, `sigma ~ 0 + z`, `sigma ~ f`, an ID shared by `mu` and `sigma`, `shape ~`, `phi ~`, `zi ~` (zero-inflated negative binomial); five non-linear models (group terms, a factor in a parameter, `inv_logit`, powers, unary minus); `mvbind()` with `set_rescor(TRUE)` and `(FALSE)` and groups, `mi()` with its sub-model; and the families: Bernoulli (logit, probit), Student-t, `brmsfamily("Gamma")`, Poisson with groups, cumulative with groups, with an ordered-factor response and with a response coded 1, 2, 4 (brms's `1..max(y)`, matched), categorical (text and integer codes), a two-Gaussian mixture, zero-inflated and hurdle Poisson.

Refused by name (113, `capability:` in the test ids), by area:

- *Population-level*: an undeclared factor, pandas Categorical, ordered factor or logical column (the message says to declare `categorical=`); `bf(cmc = FALSE)`; interactions without both main effects (`x:z`, `f:f2`, `x + x:f`, `f + x:f`); three-way; `- z`; a duplicated term or interaction; missing values in a predictor or the response (brms drops the rows with a warning); `I(x * z)`, `I(xp^0.5)`, `I(x > 0)`, `log(xp + 1)`, `abs()`, `poly()`, `scale()`, `factor()`; two offsets; `offset(2 * off)`.
- *Group-level*: `(0 + x | g)`; the same term twice; a slope without its population-level term (D8); factor, transform, interaction and `mo()` slopes; three grouping columns; `|p|` on one term; `gr()`'s `cor`, `id`, `dist`, `cov`; `mm()`'s `scale = FALSE`, slopes and `mmc()`.
- *Addition*: `se()` (three forms), `weights(scale = TRUE)`, `trials(15)`, text censoring codes, per-row `trunc()` bounds, a count's lower bound, `cens() + trunc()`, `rate()`, `y | mi()`, `mi(sdy)`, `index()`, `subset()`, `thres()`, `thres(gr = )`, `cat()`, `dec()`, `vreal()`, `vint()`.
- *Special*: `s()`, `t2()` (generic message), `gp()` with two inputs, `by`, another kernel or `scale = FALSE`; `mo():x`, `mo(id = )`; `me(gr = )`, `me() * z`; `cs()` in `cumulative`, `cs(x + z)`, `(cs(1) | g)`.
- *Autocorrelation*: `cov = TRUE`, `cosy()`, `unstr()`, `car()`, `sar()`, `fcor()`, `bf(autocor = )`.
- *`bf()`*: `sparse`, `decomp`, `unused`, a fixed `sigma = 2` (each a `TypeError` naming the keyword), `loop = FALSE`, `mu ~ x`.
- *Distributional, non-linear, multivariate*: `sigma ~` with a group term, an offset or `s()`; `nu ~`; `zi ~` for the zero-inflated Poisson; `a + b ~ 1`; a non-linear Poisson model; `sigma ~` in a non-linear model; `mvbind()` without `set_rescor()` (D58); an ID across responses; one formula per response (`bf() + bf()`, `mvbf()`, `subset()`); addition terms after `mvbind()`.
- *Families*: `poisson("identity")`, a logical Bernoulli response, categorical with groups, multinomial, Dirichlet, a mixture with `mu2 ~` or a Student-t component, Wiener, custom families, `cox`.

brms itself accepts some things quietly that AutoBayes refuses, which is the better behaviour: `cbind(y, y2) ~ x` gives brms a univariate Gaussian model with a two-column `Y` (its help page warns that `cbind()` is "reserved for other purposes"), and `y ~ x + x` and `(1 | g) + (1 | g)` are silently merged.

brms	AutoBayes	deliberate?	what a brms user trips over
`brm(formula, data, family = , prior = )`: build, compile and sample in one call	`m = ab.<family>(formula, data, priors, **options)`, then `ab.fit(m, ...)` (or `m.fit(...)`)	design (no row)	two steps; the family is the function name, not an argument (`ab.Model(..., family=)` exists)
`prior = NULL`: defaults filled in	`priors` is required; `ab.default_priors(formula, data, family=)` returns brms's defaults	D1	"`gaussian() missing 1 required positional argument: 'priors'`"
`set_prior("normal(0, 1)", class = "b", coef = "x")`, `prior(normal(0, 1), class = b)`	`ab.Priors(b={"x": ab.Normal(0, 1)})`: one keyword per class, mappings by coefficient, factor or parameter (`sd={"g": ...}`, `sigma={"intercept": ..., "b": ...}`, `nlpars=`, `resp=`)	design (no row)	strings are refused ("`b must be a Prior or a mapping`"); a class-wide prior plus one coefficient's own is not possible: a `b` mapping must cover every term ("nothing is broadcast"); `class = sd` without `group` must be keyed by factor once there are two factors; `class = cor` likewise (D20)
any Stan density, `lb`/`ub`, `constant()`	16 constructors: `Normal`, `StudentT`, `HalfNormal`, `HalfStudentT`, `HalfCauchy`, `Exponential`, `Gamma`, `InverseGamma`, `Beta`, `Logistic`, `Dirichlet`, `LKJ`, `Uniform` (`ndt` only), `Flat`, `Horseshoe`, `R2D2`	partly (D22, D29, D78, D118)	brms's `normal(0, 2)` on an SD is `HalfNormal(2)`; no `Cauchy`, `lognormal`, `constant()` or bounds; arguments are positional in Stan's order (`StudentT(nu, mu, sigma)`), keywords are PyMC's (`sigma=`, not `sd=`)
`get_prior()`, `default_prior()`: a table	`ab.default_priors()` returns a `Priors`; `m.prior_table()` is brms's table (`prior`, `class`, `coef`, `group`, ..., `source`)	D1	the table needs a built model
`bf(y ~ x, sigma ~ z, nl = , center = )`; `lf()`, `nlf()`, `set_nl()`, `mvbf()`, `set_rescor()`	a list of formula strings; keywords `nl=`, `loop=`, `center_predictors=`, `quantile=`, `rescor=`	partly (D34, D58, D63)	`cmc`, `sparse`, `decomp`, `unused`, `autocor`, fixed parameters: no spelling; `rescor` has no default (D58)
`family = poisson()`, `brmsfamily("Gamma")`, `Gamma()`, `bernoulli("probit")`	`ab.poisson`, `ab.gamma(..., link="inverse")`, `ab.bernoulli(..., link="probit")`	D9, D19	`stats::Gamma()` is inverse, `ab.gamma` log (D9 misstates brms's own default, Section 3)

Table 3: The call surface, part 1: building a model and its priors. "no row": the difference is deliberate in the library's design but has no divergence row.

brms	AutoBayes	deliberate?	what a brms user trips over
factors and characters are coded automatically	`categorical={column: reference}` is required; `str`, `bool` and `category` columns are refused without it	design (M3a-1; no row)	an ordered factor is then treatment-coded, not `contr.poly` (silent)
rows with `NA` are dropped with a warning	refused: "missing values in columns [...]; M0 does not handle missing data"	no row	drop the rows yourself; the message's "M0" is a milestone name
`data2 = list(M = ...)`	none	D69, D105	`car()`, `sar()`, `fcor()`, `gr(cov = )` are refused
`chains = 4, iter = 2000, warmup = iter/2, thin, cores, seed, control = list(adapt\_delta = 0.8, max\_treedepth = 10), init, backend`	`ab.fit(m, backend="pymc", device="cpu", chains=4, draws=1000, tune=1000, seed=None, target_accept=0.9, cores=None, chain_method=None, **sample_kwargs)`	no row	`draws` are the kept draws (brms's `iter` includes warm-up); `target_accept` defaults to 0.9, brms's `adapt_delta` to 0.8; no `thin`; other keywords are passed on to Bambi's and PyMC's sampler (not checked here)
`sample_prior = "yes", "only"`	`ab.prior_draws(m)`, `ab.prior_predictive(m)`	D124	prior draws are drawn after the fact, not stored in the fit
a `brmsfit`; `print()`, `summary()`	a `FitResult`: ArviZ `idata`, `report()` of the sampler run	no row	the posterior's names are Bambi's and PyMC's (`Intercept`, `x`, `1|g`, `1|g_sigma`), not brms's (`b_Intercept`, `sd_g__Intercept`, `r_g[g1,Intercept]`); rows D21, D43, D59, D75 cover some

Table 4: The call surface, part 2: data, sampling and results.

brms	AutoBayes	notes
`posterior_epred`	`ab.predict(m, fit, kind="mean")`	draws; binomial: `p`, with `expected = trials p` (D5); `dpar=`, `nlpar=`
`posterior_linpred`	`kind="linear"`	
`posterior_predict`	`kind="response"`	`re_formula = NA` is `include_group_effects=False`; `allow_new_levels`/`sample_new_levels` are `new_groups=` (refuses by default, D6)
`fitted`, `predict` (summaries)	none	summarise the draws yourself
`residuals`, `predictive_error`	none	
`log_lik`, `loo`, `waic`	`ab.log_lik`, `ab.loo`, `ab.waic`	fitted rows only, no `newdata`, `moment_match` or `reloo` (D48)
`loo_compare`, `model_weights`	`ab.loo_compare({name: loo, ...})`, `ab.model_weights(results, weights)`	`weights` has no default; stacking, brms's default, is built and withheld (D92, D126)
`loo_model_weights`, `kfold`, `loo_R2`	none	D48, D126
`bayes_R2`	`ab.bayes_R2`	per draw (brms's `summary = FALSE`)
`hypothesis`	`ab.hypothesis(m, fit, "x > 0", prior=)`	brms's coefficient names; class `b` only (D97)
`conditional_effects`	`ab.conditional_effects(m, fit, ["x"])`	tables, no plots; effects must be named (D98)
`prior_draws`, `prior_summary`	`ab.prior_draws(m)`; `m.prior_table()`, `m.describe()`	
`fixef`, `ranef`, `coef`, `VarCorr`, `summary`, `as_draws_df`, `ngrps`, `nobs`	none	`fit.idata` with ArviZ (`az.summary`); names differ from brms's (Section 4)
`pp_check`, `plot`, `mcmc_plot`	none	no plotting
`stancode`, `standata`	`m.describe()`, `m.pymc_model`, `m.design`	
`update`, `add_criterion`, `bridge_sampler`, `bayes_factor`, `post_prob`, `emmeans`	none	

Table 5: Post-processing. The names that exist are brms's (`ab.<name>`); the outputs are draws or pandas tables rather than brms's printed summaries.

item	what it is; does the library depend on it	recommendation
family constructors, `Priors` and the prior constructors, `default_priors`, `fit`, `predict` and the post-processing functions, `FormulaError`, `PriorError`	the brms-like surface	keep
`Model`, `MultivariateModel`, `NonlinearModel`, `FitResult`, `Loo`, `Waic`, `Hypothesis`, `Prior`, `DeviceError`	the types these return or raise	keep
`environment`, `format_environment`, `fit_record`, `compare_records`, `format_comparison` (`session.py`)	the resumable-session records of the agent scaffold: what was fit, on what, with which seed; nothing in the library calls them (`nonlinear.py` has a hook for `fit_record`); 50 test files and the milestone examples use them	move under `autobayes.session` (still importable), drop from `__all__` after a deprecation
`panel` (`diagnostics.py`)	a diagnostic panel with thresholds from `reference/diagnostics.md`; used by `fit_record` only	with `session`
`ModelSpec`, `PopTerm`, `GroupTerm`, `Categorical`, `parse_formula`, `NonlinearSpec`	the term model's internals; used inside the library and by tests, by no example	leave in `autobayes.formula`, drop from `__all__`
import-time JAX settings (`__init__.py`)	`import autobayes` enables float64 and sets NumPyro's host device count for the whole process	keep (the comment's reason holds), document in the README
milestone jargon in output	`describe()` prints "AutoBayes M0 model" (M1, M2 for Bernoulli and binomial; the family `tag`); "M0 does not handle missing data"; "this slice"/"this subset" in 36 of 132 corpus refusals, 8 citing `docs/m*-decision.md`	rewrite: the family's name, "not supported", current documentation pages

Table 6: What the package exports and prints beyond a brms-like library (`ab.__all__` has 93 names).

item	what it is; does the library depend on it	recommendation
`SKILL.md`, `skills/`, `install.sh`, `bin/autobayes-init`, `scaffold/`, `reference/`, `cluster/` (41 files)	the agent-scaffold and pre-research pack (`docs/agent-scaffold.md`); the library reads none of it; two strings cite `reference/*.md` (`diagnostics.py`'s docstring, a `fit()` note)	move to a repository of its own (or one `agent-scaffold/` directory); your decision
`docs/*-evidence/` (17 directories, 365 files, 2.8 MB)	milestone evidence; `tests/test_sampling.py` and `tests/test_evidence_provenance.py` check `docs/m0-evidence` and `docs/m1-evidence` byte for byte; a comment in `likelihoods.py` cites `docs/m2-evidence`	move under `docs/history/` with those tests' paths
`docs/*-decision.md` (17), `docs/milestones.md`, `docs/why-autobayes.md`, `docs/m0-gpu-validation.md`	milestone records; 11 citations in library error messages and docstrings	`docs/history/`; re-point the citations
`docs/coordination/` (`STATUS.md`, `NEXT_MILESTONES_PROPOSAL.md`)	agent coordination state; nothing depends on it	`docs/history/` or out of the repository
`examples/m*.py` (9)	milestone examples built on `fit_record`, `compare_records`	`examples/history/`; `examples/brms_parity/` is the current tour
`tests/evidence.py`, `test_evidence_provenance.py`, `test_session.py`, `test_fresh_process.py`, `test_diagnostics_panel.py`	tests of the above	move with what they test
`autobayes/identity.py`	the model's frozen target identity and the fit binding: `predict()`, `log_lik()` and `hypothesis()` refuse a fit of another model; not in brms	keep (a safety check, not scaffold)
`oracle/`, `tests/oracle/`, `tools/`	the brms oracle and the parity loop's tooling	keep

Table 7: What the repository ships beyond a brms-like library. Nothing was deleted.

4 The call surface

5 Post-processing names

6 Extraneous things

7 Recommendations, ranked

1. **Ordered factors** (the one silent misreading; small). Refuse `categorical=` on an ordered pandas Categorical with a message naming brms's polynomial contrasts, or build `contr.poly` columns and freeze a G1 case. The refusal is a few lines in `formula.py`; the coding is a small feature.
2. **Correct D9 and `ab.gamma`'s docstring** (minutes): brms's default Gamma link is `log`; only `stats::Gamma()` is `inverse`, so a brms user who wrote `family = Gamma()` needs `link="inverse"`. Also D20 still says `gr()` is refused; `gr(g)` and `gr(g, by = )` are taken since D08.
3. **The 16 unclear refusals** (small). Accept `(x | g)` and `(x || g)` as `(1 + x | g)` and `(1 + x || g)`, which they are in brms (the same model; no new likelihood); fix the advice for `(1 | g) + (0 + x | g)` to `(1 + x || g)` and for a shared ID to one term; give `0 + Intercept`, `::`, `^`, `%in%`, `.`, backticks, transformed and `cbind()` responses, `(x - 1 | g)`, `disc` and `nlf()` messages that name them; give `s()`/`t2()` their own "smooths are not offered" message. Each is a message in `formula.py` plus a re-run of the audit.
4. **Cheap brms spellings** of constructs already offered (small each, an oracle case where the likelihood is touched): `gr(g, cor = FALSE)` as `||`; `gr(g, id = "p")` as `|p|` where `|p|` is taken; `trials(<number>)`; `rate(denom)` for the Poisson family (`offset(log(denom))` there; brms notes it is not an offset for the negative binomial); brms's text censoring codes; `cat(K)`/`thres(K)` when `K` is the data's.
5. **Project jargon out of user-facing text** (small to medium; tests that pin messages move with it): the family's name instead of `M0`/`M1`/`M2` in `describe()`, "not supported" instead of "this slice", current documentation pages instead of milestone decision documents. Keep the divergence ids.
6. **Divergence rows for the undocumented call-surface differences** (small): the required `categorical=` and its treatment coding, priors that are never broadcast, refused missing values, `fit()`'s `draws`/`tune` and `target_accept = 0.9`, the posterior's names.
7. **The top-level namespace** (medium): `session`, `panel` and the term-model names out of `__all__`, importable from their modules, with one release of deprecation warnings.
8. **brms-named draws** (medium to large): an `as_draws_df`-like export with brms's names (the oracle adapter already maps every parameter by role), then `fixef`, `ranef`, `VarCorr` and `summary` on top. This is what a brms user will look for first after `posterior_epred`.
9. **The repository** (medium, your decision): the scaffold to its own repository; the milestone history under `docs/history/`, with the two evidence tests' paths and the 11 citations re-pointed.
10. **Capabilities by likely demand** (large): `s()`/`t2()`, `se()` for meta-analysis, group-level terms in categorical and distributional formulas, a formula per response. These are on the roadmap already.

Reproducing

From the repository root: `Rscript tests/brms_syntax/brms_reading.R` (brms's side, about 25 s, needs R and brms 2.23), `python -m tests.brms_syntax.audit` (AutoBayes's side, about 12 s),

`pytest tests/test_brms_syntax_corpus.py` (250 tests, 5 s, no R).